Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Самсонов И.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 01.10.26

Москва, 2026

Постановка задачи

Вариант 18.

Родительский процесс создаёт два дочерних процесса. Первые две строки ввода задают имена файлов для результатов первого и второго дочерних процессов. Нечётные строки родитель направляет первому процессу через pipe1, чётные — второму процессу через pipe2. Дочерние процессы удаляют из строк гласные латинские буквы и записывают результат в назначенные файлы.

Общий метод и алгоритм решения

Решение состоит из двух программ: parent и child. Родитель читает имена выходных файлов, создаёт два неименованных канала и два дочерних процесса. Каждый потомок перенаправляет стандартный ввод на читающий конец своего канала, стандартный вывод — в соответствующий файл, после чего запускает программу child.

Использованные системные вызовы:

- pipe — создание однонаправленных каналов;
- fork — создание дочерних процессов;
- open — создание и открытие выходных файлов;
- dup2 — перенаправление стандартных потоков;
- execl — замена образа дочернего процесса программой child;
- read и write — чтение и передача данных;
- close — закрытие неиспользуемых файловых дескрипторов;
- waitpid — ожидание завершения дочерних процессов;
- sigaction — настройка обработки SIGPIPE;
- _exit — завершение дочернего процесса при ошибке.

Родитель считывает пользовательский ввод посимвольно. Номер текущей строки начинается с единицы и увеличивается после символа перевода строки. Для нечётного номера символ записывается в pipe1, для чётного — в pipe2. Такой способ не ограничивает длину обрабатываемой строки размером буфера.

Программа child читает свой стандартный ввод до EOF, пропускает символы A, E, I, O, U в обоих регистрах и записывает остальные символы в стандартный вывод. После закрытия родителем записывающих концов каналов дочерние процессы получают EOF, завершаются, а родитель проверяет их статусы. Ошибки системных вызовов обрабатываются; вызовы, прерванные сигналом EINTR, повторяются.

Код программы

parent.c

```
#include <signal.h>
#include <errno.h>
#include <fcntl.h>
#include <stddef.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#define FILE_NAME_SIZE 256


int wait_for_child(pid_t child){
    int status;
    pid_t result;

    do {
    result = waitpid(child, &status, 0);
    }while (result == -1 && errno == EINTR);

    if (result == -1) {
```

```
        return -1;
    }

    if (!WIFEXITED(status)){
        return -1;
    }
    if (WEXITSTATUS(status) != 0){
        return -1;
    }
    return 0;
}

void print_error(const char *message, size_t size){
    write(STDERR_FILENO, message, size);
}

int read_file_name(char *file_name){
    int index = 0;
    char character;

    while (index < FILE_NAME_SIZE - 1){
        ssize_t bytes_read = read(STDIN_FILENO, &character, 1);

        if (bytes_read == -1){
            if (errno == EINTR){
                continue;
            }
            return -1;
        }
        if (bytes_read == 0 || character == '\n'){
            break;
        }
        file_name[index] = character;
        index++;
    }

    file_name[index] = '\0';

    if (index == 0){
        return -1;
    }
    return 0;
}

int main(void)
{

    struct sigaction action;
    action.sa_handler = SIG_IGN;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;
    if (sigaction(SIGPIPE, &action, NULL) == -1){
        const char message[] = "Error: cannot configure SIGPIPE\n";
        print_error(message, sizeof(message) -1);
        return 1;
    }

    char first_file_name[FILE_NAME_SIZE];
    char second_file_name[FILE_NAME_SIZE];

    const char first_prompt[] = "First file name: ";
    write(STDOUT_FILENO, first_prompt, sizeof(first_prompt) - 1);

    if (read_file_name(first_file_name) == -1){
        const char message[] = "Error: invalid first file name\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    const char second_prompt[] = "Second file name: ";
    write(STDOUT_FILENO, second_prompt, sizeof(second_prompt) - 1);

    if (read_file_name(second_file_name) == -1){
        const char message[] = "Error: invalid second file name\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    int pipe2[2];
    int pipe1[2];

    if (pipe(pipe1) == -1) {
        const char message[] = "Error: cannot create pipe1\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    if (pipe(pipe2) == -1) {
        const char message[] = "Error: cannot create pipe2\n";
        print_error(message, sizeof(message) - 1);

        close(pipe1[0]);
        close(pipe1[1]);
        return 1;
    }

    pid_t child1 = fork();

    if (child1 == -1){
        const char message[] = "Error: cannot create child1\n";
        print_error(message, sizeof(message) - 1);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
```

```
    close(pipe2[1]);

    return 1;
}

if (child1 == 0){
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    int output_file = open(first_file_name, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_file == -1){
        const char message[] = "Error: child1 cannot open output file\n";
        print_error(message, sizeof(message) - 1);
        close(pipe1[0]);
        _exit(1);
    }

    if (dup2(pipe1[0], STDIN_FILENO) == -1) {
        const char message[] ="Error: child1 cannot redirect stdin\n";
            print_error(message, sizeof(message) - 1);
        close(pipe1[0]);
        close(output_file);
        _exit(1);
    }

    if(dup2(output_file, STDOUT_FILENO) == -1){
        const char message[] ="Error: child1 cannot redirect stdout\n";
            print_error(message, sizeof(message) - 1);

        close(pipe1[0]);
        close(output_file);
        _exit(1);
    }
    close(pipe1[0]);
    close(output_file);
    execl("./child", "child", (char *)0);
    const char message[] = "Error: cannot execute child\n";
    print_error(message, sizeof(message) - 1);
    _exit(1);
}

pid_t child2 = fork();

if (child2 == -1){
    const char message[] = "Error: cannot create child2\n";
    print_error(message, sizeof(message) - 1);

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(child1, NULL, 0);
    return 1;
}

if (child2 == 0){
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);

    int output_file = open(second_file_name, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_file == -1){
        const char message[] = "Error: child2 cannot open output file\n";
        print_error(message, sizeof(message) - 1);
        close(pipe2[0]);
        _exit(1);
    }

    if (dup2(pipe2[0], STDIN_FILENO) == -1){
        const char message[] = "Error: child2 cannot redirect stdin\n";
        print_error(message, sizeof(message) - 1);
        close(pipe2[0]);
        close(output_file);
        _exit(1);
    }

    if (dup2(output_file, STDOUT_FILENO) == -1){
        const char message[] ="Error: child2 cannot redirect stdout\n";
        print_error(message, sizeof(message) - 1);
        close(pipe2[0]);
        close(output_file);
        _exit(1);
    }

    close(pipe2[0]);
    close(output_file);
    execl("./child", "child", (char *)0);
    const char message[] = "Error: cannot execute child\n";
    print_error(message, sizeof(message) - 1);
    _exit(1);
}

close(pipe1[0]);
close(pipe2[0]);

char character;
int line_number = 1;

while (1){
    ssize_t bytes_read = read(STDIN_FILENO, &character, 1);
```

```
        if (bytes_read == -1){
            if (errno == EINTR){
                continue;
            }
            const char message[] = "Error: cannot read input\n";
            print_error(message, sizeof(message) - 1);
            close(pipe1[1]);
            close(pipe2[1]);

            waitpid(child1, NULL, 0);
            waitpid(child2, NULL, 0);
            return 1;
        }
        if (bytes_read == 0){
            break;
        }
        int destination;

        if (line_number % 2 == 1){
            destination = pipe1[1];
        } else{
            destination = pipe2[1];
        }
        ssize_t bytes_written;
        do {
            bytes_written = write(destination, &character, 1);
        } while (bytes_written == -1 && errno == EINTR);

        if (bytes_written != 1) {

        const char message[] = "Error: cannot write to pipe\n";
        print_error(message, sizeof(message) - 1);
        close(pipe1[1]);
        close(pipe2[1]);

        waitpid(child1, NULL, 0);
        waitpid(child2, NULL, 0);
        return 1;
        }
        if (character == '\n'){
            line_number++;
        }
    }

    close(pipe1[1]);
    close(pipe2[1]);

    int child1_result = wait_for_child(child1);
    int child2_result = wait_for_child(child2);

    if (child1_result == -1 || child2_result == -1){
        const char message[] = "Error: child process failed\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    return 0;
}
```

child.c

```
#include <errno.h>
#include <unistd.h>

int is_vowel(char character){
    return character == 'a' ||
    character == 'e' ||
     character == 'i' ||
     character == 'o' ||
     character == 'u' ||
     character == 'A' ||
     character == 'E' ||
     character == 'I' ||
     character == 'O' ||
     character == 'U';
}

int main(void){
    char character;

    while(1){
        ssize_t bytes_read = read(STDIN_FILENO, &character, 1);

        if (bytes_read == -1){
            if (errno == EINTR){
                continue;
            }

            const char message[] = "Error: child cannot read\n";
            write(STDERR_FILENO, message, sizeof(message) - 1);
            return 1;
        }

        if (bytes_read == 0){
            break;
        }
        if (is_vowel(character)){
            continue;
        }

        ssize_t bytes_written;
        do {
            bytes_written = write(STDOUT_FILENO, &character, 1);
        } while(
            bytes_written == - 1 && errno == EINTR);

        if (bytes_written != 1) {
```

```
            const char message[] = "Error: child cannot write\n";
            write(STDERR_FILENO, message, sizeof(message) - 1);
            return 1;
        }
    }
    return 0;
}
```

Протокол работы программы

Программа собрана компилятором cc с включёнными предупреждениями. Для проверки введены четыре строки: нечётные должны попасть в odd.txt, чётные — в even.txt.

Тестирование:

```
        $ cc -std=c11 -Wall -Wextra -Wpedantic -Werror child.c -o child
$ cc -std=c11 -Wall -Wextra -Wpedantic -Werror parent.c -o parent
$ ./parent
First file name: odd.txt
Second file name: even.txt
Operating Systems
Queue audio
Process communication
Fork and pipe
<Ctrl-D>
$ cat odd.txt
prtng Systms
Prcss cmmnctn
$ cat even.txt
Q d
Frk nd pp
```

Трассировка системных вызовов

Трассировка получена при запуске исходных parent.c и child.c в Alpine Linux 3.22 под Linux ARM64 в Docker. Программы собраны GCC с флагами -std=c11 -Wall -Wextra -Wpedantic -Werror; для parent также задан -D_POSIX_C_SOURCE=200809L. Для короткого теста использованы файлы odd.txt и even.txt и две строки Apple и Queue. В odd.txt записано ppl, в even.txt — Q; переводы строк сохранены.

Команда запуска

```
printf "odd.txt\neven.txt\nApple\nQueue\n" | \
strace -f -qq -s 100 \
  -e trace=process,read,write,close,pipe,pipe2,dup2,dup3,\
open,openat,rt_sigaction -o trace.log ./parent-alpine
```

Параметр -f включает дочерние процессы, -o сохраняет журнал в trace.log, -e trace оставляет выбранные системные вызовы, -qq убирает служебные сообщения strace. Первое число в строке журнала — PID процесса. Вызов записан как имя(аргументы) = результат. Для read и write результат равен числу обработанных байтов; read = 0 означает конец ввода.

В этом запуске родитель имеет PID 31, первый ребёнок — 32, второй — 33. Ниже оставлены вызовы, относящиеся к алгоритму лабораторной; загрузка библиотек и посимвольное чтение имён файлов исключены. Записи сгруппированы по процессам, поэтому порядок между группами не отражает общий порядок выполнения. Разорванные strace строки unfinished/resumed объединены без изменения аргументов и результатов.

Родительский процесс

```
31 rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[], sa_flags=SA_RESTORER,
sa_restorer=0xffff995fba2c}, NULL, 8) = 0
31 pipe2([3, 4], 0)                     = 0
31 pipe2([5, 6], 0)                     = 0
31 clone(child_stack=NULL, flags=SIGCHLD) = 32
31 clone(child_stack=NULL, flags=SIGCHLD) = 33
31 close(3 )                = 0
31 close(5 )                = 0
31 read(0,  "A", 1)           = 1
31 write(4, "A", 1 )                 = 1
31 read(0,  "p", 1)           = 1
31 write(4, "p", 1 )                 = 1
31 read(0, "p", 1)                      = 1
31 write(4, "p", 1 )                = 1
```

```
31 read(0,  "l", 1)         = 1
31 write(4, "l", 1 )              = 1
31 read(0,  "e", 1)         = 1
31 write(4, "e", 1)                 = 1
31 read(0,  "\n", 1)        = 1
31 write(4, "\n", 1 )              = 1
31 read(0,  "Q", 1)         = 1
31 write(6, "Q", 1 )              = 1
31 read(0,  "u", 1)         = 1
31 write(6, "u", 1 )              = 1
31 read(0,  "e", 1)         = 1
31 write(6, "e", 1 )              = 1
31 read(0,  "u", 1)         = 1
31 write(6, "u", 1)                 = 1
31 read(0, "e", 1)                  = 1
31 write(6, "e", 1)                 = 1
31 read(0, "\n", 1)                 = 1
31 write(6, "\n", 1)                = 1
31 read(0, "", 1)                   = 0
31 close(4)                         = 0
31 close(6)                         = 0
31 wait4(32,  0xffffd9f2ffc0, 0, NULL) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
31 wait4(32, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 32
31 wait4(33, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 33
31 exit_group(0)                    = ?
```

В этом запуске pipe1[0] = 3, pipe1[1] = 4, pipe2[0] = 5, pipe2[1] = 6. Родитель передаёт нечётную строку через write(4, ...), чётную — через write(6, ...). Дескрипторы 3 и 5 нужны детям для чтения из соответствующих каналов.

rt_sigaction задаёт игнорирование SIGPIPE. Два pipe2 создают каналы: первый элемент каждой пары — конец чтения, второй — конец записи. clone создаёт детей. Родитель закрывает свои концы чтения и направляет символы Apple с переводом строки в первый канал, а Queue — во второй. После EOF он закрывает концы записи. wait4 получает статусы завершения детей; WEXITSTATUS равен 0 означает успешное завершение. exit_group завершает родителя.

Первая попытка wait4 помечена ERESTARTSYS: ожидание прервано сигналом о завершении другого ребёнка. Это внутренняя отметка ядра для возобновления вызова. Следующий wait4 успешно возвращает PID ожидаемого ребёнка; затем получен статус второго. В журнале оба статуса содержат WIFEXITED и WEXITSTATUS == 0.

Дочерний процесс 1

```
32 close(4 )              = 0
32 close(5 )              = 0
32 close(6 )              = 0
32 openat(AT_FDCWD, "odd.txt", O_WRONLY|O_CREAT|O_TRUNC|O_LARGEFILE, 0644 )            = 4
32 dup3(3, 0, 0 )               = 0
32 dup3(4, 1, 0 )               = 1
32 close(3 )              = 0
32 close(4 )              = 0
32 execve("./child", ["child"], 0xffffd9f30378 /* 5 vars */ )             = 0
32 read(0, "A", 1)                  = 1
32 read(0, "p", 1)                  = 1
32 write(1, "p", 1 )              = 1
32 read(0,  "p", 1)         = 1
32 write(1, "p", 1 )              = 1
32 read(0,  "l", 1)         = 1
32 write(1, "l", 1 )              = 1
32 read(0,  "e", 1)         = 1
32 read(0,  "\n", 1)        = 1
32 write(1, "\n", 1 )              = 1
32 read(0, "", 1)                   = 0
32 exit_group(0 )         = ?
```

Ребёнок закрывает ненужные концы каналов. openat открывает odd.txt для записи с созданием и очисткой файла. dup3 перенаправляет дескриптор 0 на первый канал, а 1 — в odd.txt. После закрытия исходных дескрипторов перенаправленные 0 и 1 продолжают работать. execve загружает child в тот

же процесс. Вызовы read получают A, p, p, l, e и перевод строки; write записывает только p, p, l и перевод строки. Гласные A и e пропускаются. После read = 0 ребёнок завершается с кодом 0.

Дочерний процесс 2

```
33 close(3 )              = 0
33 close(4 )              = 0
33 close(6 )              = 0
33 openat(AT_FDCWD, "even.txt", O_WRONLY|O_CREAT|O_TRUNC|O_LARGEFILE, 0644 )            = 3
33 dup3(5, 0, 0 )               = 0
33 dup3(3, 1, 0 )               = 1
33 close(5 )              = 0
33 close(3 )              = 0
33 execve("./child", ["child"], 0xffffd9f30378 /* 5 vars */ )             = 0
33 read(0,  "Q", 1)         = 1
33 write(1, "Q", 1 )               = 1
33 read(0,  "u", 1)         = 1
33 read(0, "e", 1)                    = 1
33 read(0,  "u", 1)         = 1
33 read(0,  "e", 1)         = 1
33 read(0,  "\n", 1)        = 1
33 write(1, "\n", 1 )              = 1
33 read(0,  "", 1)          = 0
33 exit_group(0)                      = ?
```

Второй ребёнок аналогично перенаправляет ввод на второй канал, а вывод — в even.txt. Он читает Queue и перевод строки. Буква Q является согласной: write записывает её в файл. Гласные u, e, u, e пропускаются, затем отдельный write сохраняет перевод строки. read = 0 завершает цикл; exit_group(0) завершает процесс успешно.

Связь исходного кода и трассировки

В этой Linux-среде функции C обращаются к ядру под следующими именами: pipe → pipe2, fork → clone, open → openat, dup2 → dup3, execl → execve, waitpid → wait4, sigaction → rt_sigaction. Обычное завершение main приводит к exit_group. Имена в журнале могут отличаться на другой архитектуре или при другой реализации стандартной библиотеки. execl является библиотечной оболочкой, а наблюдаемый системный вызов запуска программы — execve.

Стандартные дескрипторы: 0 — ввод, 1 — вывод, 2 — ошибки. Номера остальных дескрипторов действуют внутри конкретного процесса: одинаковое число у разных процессов само по себе не доказывает общий файл. В этой программе после fork дети наследуют дескрипторы каналов родителя. close удаляет только указанную ссылку процесса; EOF на чтении канала возникает после закрытия всех его концов записи и чтения оставшихся данных.

Проверка результата

```
odd.txt:  "ppl\n"
even.txt: "Q\n"
```

Трассировка показывает создание двух детей, независимое перенаправление ввода и вывода, передачу строк через два канала и успешное завершение всех трёх процессов. Результаты соответствуют удалению латинских гласных; символы перевода строки сохраняются.

Документация strace: https://github.com/strace/strace/blob/master/doc/strace.1.in

Вывод

В ходе лабораторной работы была реализована передача строк между тремя процессами через два неименованных канала. Родительский процесс корректно распределяет строки по чётности их номера, а дочерние процессы удаляют гласные и записывают результаты в разные файлы. Основная сложность состояла в правильном закрытии неиспользуемых концов каналов и перенаправлении дескрипторов через dup2.

Обработка EOF, EINTR, SIGPIPE и статусов waitpid обеспечивает корректное завершение программы и диагностику ошибок.